Best Backend Architecture Plan

Project: NetOps AI — Real-Time Network Monitoring, Automation & Reliability Backend

This backend should behave like a mini **Cisco Catalyst Center-style system**: centralized telemetry, device health, automation, assurance, config management, reporting, and APIs. Catalyst Center supports telemetry, assurance, automation events, APIs, and external notification handlers, which matches this project direction. (Cisco DevNet) It also aligns with the uploaded project notes about centralized dashboards, AI troubleshooting, automation, APIs, and reporting.

---

1. Final Backend Architecture

Simulated Devices
 Routers / Switches / APs / Edge Nodes
 ↓
Telemetry Agent Service
 Python async producers
 ↓
Redis Streams / Kafka
 real-time event ingestion
 ↓
Telemetry Ingestion Service
 FastAPI + workers
 ↓
Rules Engine + AI Anomaly Engine
 thresholds + ML detection
 ↓
Automation Engine
 restart / rollback / alert / incident creation
 ↓
PostgreSQL + TimescaleDB + Redis
 configs, devices, incidents, time-series metrics
 ↓
Notification Service
 WebSocket + email/webhook
 ↓
Reporting Service

PDF/CSV/JSON reports
↓
Frontend / External APIs
React dashboard / integrations

---

2. Recommended Backend Stack

Area	Best Choice
API Backend	**FastAPI**
Streaming	**Redis Streams first**, Kafka later
Time-Series DB	**TimescaleDB**
Main DB	**PostgreSQL**
Cache	**Redis**
Background Jobs	**Celery + Redis**
AI/ML	**Scikit-learn Isolation Forest**
Real-Time Updates	**FastAPI WebSockets**
Observability	**Prometheus + Grafana + OpenTelemetry**
Reports	**ReportLab / WeasyPrint**
Deployment	**Docker Compose**

Use **Redis Streams first** because it is simpler for a student/portfolio project and good for moderate real-time workloads; Kafka is better when scaling to very high-volume durable streams. (OneUptime)

Use **TimescaleDB** because network telemetry is time-series data: CPU, memory, latency, packet loss, uptime, bandwidth, and event timestamps. TimescaleDB is designed for IoT/monitoring-style high-throughput time-series workloads. (OneUptime)

---

3. Backend Services

A. Device Simulator Service

Generates fake network devices.

device-simulator/

Responsibilities:

- simulate routers, switches, APs, edge nodes
- send telemetry every 2–5 seconds
- simulate failures
- simulate config drift
- simulate packet loss and latency spikes

Example event:

```
{
  "device_id": "router-101",
  "device_type": "router",
  "cpu": 91,
  "memory": 78,
  "latency_ms": 245,
  "packet_loss": 6.2,
  "bandwidth_mbps": 820,
  "status": "warning",
  "timestamp": "2026-05-17T12:00:00Z"
}
```

B. Telemetry Ingestion Service

telemetry-service/

Responsibilities:

- consume telemetry from Redis Streams/Kafka
- validate event schema
- store raw metrics in TimescaleDB
- calculate live device health

- publish alerts when thresholds break

C. Device Inventory Service

device-service/

Responsibilities:

- register devices
- store device metadata
- track online/offline status
- expose device APIs
- support filtering by type, location, status

Entities:

Device
Location
DeviceGroup
DeviceStatus

D. Configuration Management Service

config-service/

Responsibilities:

- store approved baseline config
- store current config
- detect config drift
- support config versioning
- rollback to previous config
- simulate config push

Important feature:

Expected config != Current config → Drift detected → Automation triggered

---

E. Rules + AI Anomaly Detection Service

anomaly-service/

Two detection layers:

1. Rule-Based Detection

Example:

CPU > 90% → Warning
Packet loss > 5% → Critical
Latency > 200ms → Degraded
No heartbeat for 30 sec → Offline

2. AI-Based Detection

Use:

- Isolation Forest for anomaly detection
- later LSTM for time-series forecasting

This detects unusual patterns, not only fixed thresholds.

F. Automation Engine

automation-service/

This is the most important role-matching component.

Responsibilities:

- auto-create incident
- auto-restart simulated device agent
- auto-run config validation
- auto-trigger rollback
- auto-send notification
- auto-mark device as recovered

Example workflow:

High packet loss detected
↓
Create incident
↓
Check config drift
↓
If drift exists → rollback config
↓
Notify dashboard
↓
Generate recovery log

G. Incident Management Service

incident-service/

Responsibilities:

- create incidents
- assign severity
- track affected devices
- store incident timeline
- resolve incidents
- generate root cause summary

Incident states:

OPEN → INVESTIGATING → MITIGATED → RESOLVED

H. Notification Service

notification-service/

Responsibilities:

- WebSocket live alerts
- email alerts
- webhook alerts
- dashboard event feed

WebSocket channels:

/ws/devices
/ws/alerts
/ws/incidents
/ws/topology

I. Reporting Service

reporting-service/

Responsibilities:

- uptime report
- SLA report
- device health report
- incident report
- config compliance report
- anomaly report

Export:

PDF
CSV
JSON

J. Observability Service

Add:

- Prometheus metrics
- Grafana dashboard
- OpenTelemetry tracing
- structured logs

This is important because the recruiter mentioned **system reliability**. OpenTelemetry, Prometheus, and Grafana are widely used for metrics, traces, and observability in distributed systems. (Grafana Labs)

---

4. Backend Database Design

PostgreSQL Tables

users
roles
devices
device_groups
device_configs
config_versions
incidents
incident_events
alerts
automation_workflows
automation_runs
reports

TimescaleDB Tables

telemetry_metrics
device_health_snapshots
network_latency_metrics
packet_loss_metrics
bandwidth_metrics

Redis Usage

live_device_status
active_alerts
websocket_sessions
task_locks
temporary health scores

---

5. API Design

Device APIs

GET /api/devices
GET /api/devices/{device_id}
POST /api/devices
PATCH /api/devices/{device_id}
GET /api/devices/{device_id}/metrics

Telemetry APIs

POST /api/telemetry
GET /api/telemetry/{device_id}
GET /api/telemetry/{device_id}/history

Config APIs

GET /api/configs/{device_id}
POST /api/configs/{device_id}
POST /api/configs/{device_id}/validate
POST /api/configs/{device_id}/rollback
GET /api/configs/{device_id}/drift

Incident APIs

GET /api/incidents
POST /api/incidents
PATCH /api/incidents/{incident_id}/resolve
GET /api/incidents/{incident_id}/timeline

Automation APIs

POST /api/automation/run
GET /api/automation/runs
GET /api/automation/workflows

Reporting APIs

GET /api/reports/uptime
GET /api/reports/incidents
GET /api/reports/config-compliance
GET /api/reports/export/pdf

---

6. Best Build Order

Phase 1 — Core Backend

Build:

- FastAPI app
- PostgreSQL models
- device CRUD
- telemetry simulator
- telemetry ingestion

Phase 2 — Real-Time Monitoring

Build:

- Redis Streams
- WebSocket alerts
- device health engine
- threshold rules

Phase 3 — Configuration Management

Build:

- config versioning
- config drift detection
- rollback simulation

Phase 4 — Automation Engine

Build:

- recovery workflows
- incident auto-creation
- alert automation

Phase 5 — AI Reliability

Build:

- anomaly detection
- reliability score
- outage risk score

Phase 6 — Reporting + Observability

Build:

- PDF/CSV reports
- Prometheus
- Grafana
- OpenTelemetry traces

---

7. Best Backend Folder Structure

```
netops-ai-backend/
│
├── services/
│   ├── api-gateway/
│   ├── device-service/
│   ├── telemetry-service/
│   ├── config-service/
│   ├── anomaly-service/
│   ├── automation-service/
│   ├── incident-service/
│   ├── notification-service/
│   └── reporting-service/
│
├── shared/
│   ├── database/
│   ├── schemas/
│   ├── auth/
│   ├── logging/
│   ├── events/
│   └── utils/
│
├── simulator/
```

```
│   └── device_simulator.py
│
├── docker-compose.yml
├── .env.example
└── README.md
```

Final Recommendation

Build this as a **modular monolith first**, then split into microservices later.

For your first real-time version:

FastAPI modular backend
+ Redis Streams
+ PostgreSQL/TimescaleDB
+ Celery workers
+ WebSockets
+ Docker Compose

That gives you a strong, realistic, production-style backend without overcomplicating the first version.